

Malware Analysis Report

Win64/Reasonix.CrackerDrv

Windows x64 Kernel-Mode Rootkit Driver — Static Analysis Findings

Classification: Windows x64 Kernel-Mode Rootkit Driver

Analyst: Static reverse engineering, cross-referenced with external threat intelligence

Report compiled by: Douglas Mun assisted by Claude Sonnet 5

Report date: 2026-07-06

Disclaimer

This report documents a **static-only** analysis of a suspected malicious binary. The sample was never executed; all findings derive from disassembly (radare2), string extraction (FLOSS), and signature checks (ClamAV) performed against the file at rest. No dynamic/behavioral analysis, sandbox detonation, or network traffic capture was performed.

Confidence ratings (High / Medium / Medium-Low / Low / Unidentified) are stated per finding and reflect the strength of the static evidence only — they are not a substitute for validation on a live or instrumented system. Several capabilities described here are opcode-gated branches of a single internal dispatcher (see Finding 3a) and remain dormant unless triggered by a specific, not fully reconstructed input; the practical exploitability and real-world trigger conditions of these branches were not independently confirmed.

This report is provided for defensive, incident response, and detection engineering purposes. It does not constitute legal, forensic-custody, or attribution advice, and no claim is made about the identity or intent of any party responsible for this sample. See **Assessment Limitations** and **Appendix: Methodology** for the full scope and boundaries of this analysis.

Table of Content

Disclaimer.....	1
Table of Content.....	2
Executive Summary.....	3
Sample Identification.....	4
Key Findings (ranked by severity).....	5
Capability Flow Diagram.....	6
Technical Analysis.....	7
1. Anti-forensics: self-deletion while loaded.....	7
2. Manual module resolution.....	8
3. Registry filtering.....	8
4. Process/thread handle manipulation — EDR-blinding.....	11
5. Direct CR3 read used for physical-to-virtual address resolution.....	13
6. Kernel-mode GUI/window subversion.....	14
7. Keyboard/mouse class-driver reconnaissance.....	15
8. Process injection / token manipulation toolkit — fully traced to three opcode-gated primitives inside the registry-callback dispatcher.....	17
External Intelligence Corroboration.....	20
Indicators of Compromise.....	21
Assessment Limitations.....	22
Recommendations (for the suspected-infected Windows host).....	23
Detection Opportunities.....	26
Glossary.....	29
Appendix: Methodology.....	33
Appendix: Corrections Made During Audit.....	34

Executive Summary

The submitted file, despite its .exe extension, is an unsigned x86-64 Windows **kernel-mode driver** compiled 2026-07-04 12:14:58 UTC — approximately two days before this analysis. Static reverse engineering confirms it is a fully-featured rootkit with eight malicious capabilities, each independently confirmed via direct disassembly (ranked by severity in Key Findings, below) — though several are not freestanding: most severely, the registry callback that anchors the driver's control flow was shown to be a single unified 92-branch opcode dispatcher (Finding 3a) whose case-gated branches include **a complete remote code-execution and token-theft toolkit** (Finding 8): kernel-mode `RtlCreateUserThread`-based remote thread injection, a chained `LoadLibraryA`-based DLL-injection primitive, four `KeStackAttachProcess/ZwAllocateVirtualMemory` shellcode-staging routines, and a raw primary-token content-swap routine capable of granting one process another's security context (e.g., privilege escalation to `SYSTEM`) — none of which run automatically at driver load; all are dormant until triggered by a specific registry operation matching the driver's internal opcode encoding. Alongside this are: process-protection/EDR-blinding via a process-handle-access-mask manipulation callback that renders a designated process untouchable by AV/EDR/Task Manager; a single genuine `mov reg, cr3` instruction feeding a physical-to-virtual address resolution routine — a real, non-standard technique, though narrower than initially reported (see Finding 5 for the corrected description); anti-forensic self-deletion from disk while resident in memory; a system-wide registry filter; manual (API-monitoring-evasive) module resolution; kernel-mode abuse of the internal win32k GUI syscall table for cross-session window enumeration and synthetic input injection; and structurally parallel (but independently implemented, not shared) reconnaissance of the keyboard and mouse class-driver stacks (`kbdclass/mouclass`), though the exact input-capture mechanism was not fully localized in the traced code.

At the time of external checks (2026-07-06), the sample had a low-but-nonzero VirusTotal detection ratio (13/70) and no MalwareBazaar family tag, consistent with a fresh, purpose-built, or recently modified tool rather than a widely circulated commodity kit. A community YARA hit against a 2017-era Winnti/ZxShell rule is a **stylistic pattern match, not a confirmed actor attribution**, and is reported here strictly as such.

Overall verdict: Malicious. Confidence: High (based on direct code-level evidence, independent of any AV/vendor labeling, which was inconsistent and largely absent at scan time).

Sample Identification

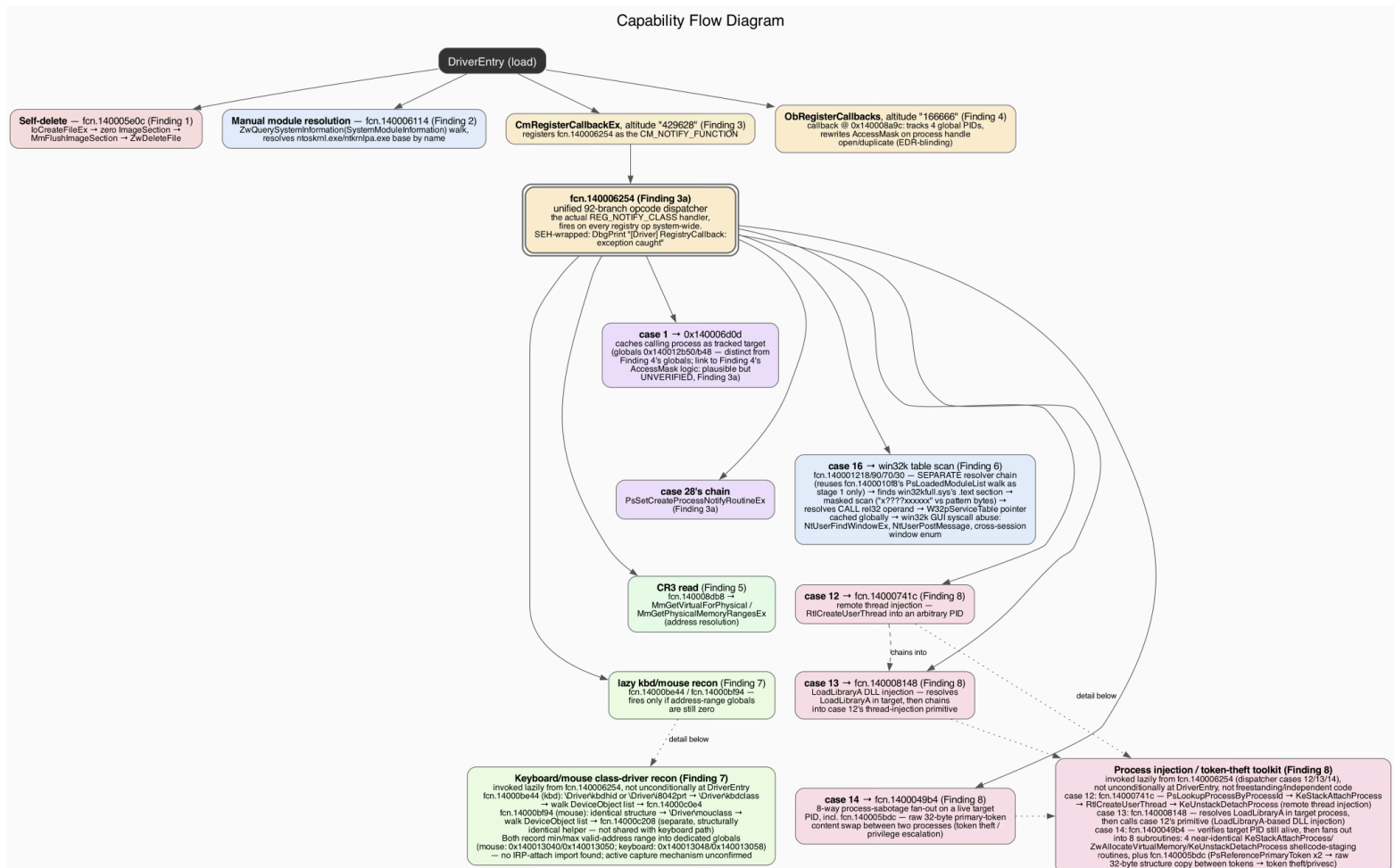
Field	Value
Filename (as received)	c7ec30029a10580a0b33b71b75d34efe053cc34d908bf75998f457a0c1f074f6.exe
SHA-256	c7ec30029a10580a0b33b71b75d34efe053cc34d908bf75998f457a0c1f074f6
ImpHash	93aaf72477167a05c85c15e6440d1c06
File type	PE32+ x86-64, IMAGE_SUBSYSTEM_NATIVE — kernel-mode driver , not a user-mode executable, despite the .exe extension
Size	74,240 bytes
Compile timestamp	2026-07-04 12:14:58 UTC
Code signing	Absent. IMAGE_DIRECTORY_ENTRY_SECURITY RVA/Size = 0 — no Authenticode signature, no signer identity
PDB / build path	\AppData\Roaming\reasonix\global-workspace\6.15\x64\Release\CrackerDrv.pdb
Imports	Exclusively ntoskrnl.exe (89 functions) and HAL.dll (1 function) — pure kernel-mode dependency surface
Notable section	fothk — non-standard, 4,096 bytes, 4,091 of 4,096 bytes are 0xCC filler, with a single relative jmp stub at offset +16 targeting 0x14000c5c0. Consistent with a reserved hook/trampoline scratch region, not a compiler-generated section
Device interface	No IoCreateDevice/IoCreateSymbolicLink import — no IOCTL/userland control surface found in this binary

An unsigned kernel driver cannot load on a stock, Secure-Boot-enforced Windows 10/11 system without either test-signing mode, Driver Signature Enforcement disabled, or a BYOVD (Bring Your Own Vulnerable Driver) exploitation chain. This sample contains none of those loader mechanisms — it is the payload, not the delivery vector.

Key Findings (ranked by severity)

1. **Remote thread/DLL injection and primary-token-swap toolkit, opcode-gated inside the registry callback** — Confidence: High. A crafted registry operation matching the driver's internal opcode encoding can trigger kernel-mode `RtlCreateUserThread` remote thread injection, a chained `LoadLibraryA`-based DLL-injection primitive, or a raw 32-byte primary-token content copy between two arbitrary processes (privilege-escalation-capable, e.g. granting a low-privilege process `SYSTEM`'s security context). See Finding 8. (*T1055 – Process Injection; T1134 – Access Token Manipulation*)
2. **Process-protection / EDR-blinding via `ObRegisterCallbacks`** — Confidence: High. Renders a designated PID immune to termination, memory access, and suspension by any other process, including security tooling. (*MITRE ATT&CK: T1562.001 – Impair Defenses: Disable or Modify Tools*)
3. **Direct CR3 read feeding a manual physical-to-virtual address translation walk** — Confidence: High for the CR3 access itself; Medium-Low for any page-table write-protection-bypass purpose. Genuine, non-standard control-register access, but paired with documented `Mm*` APIs rather than raw page-table-entry bit patching (see Finding 5 for the corrected scope). (*T1014 – Rootkit*)
4. **Anti-forensic self-deletion of the loaded driver file** — Confidence: High. Removes the on-disk artifact while remaining resident in memory. (*T1070.004 – Indicator Removal: File Deletion*)
5. **System-wide registry filtering via `CmRegisterCallbackEx`, whose notify routine is a unified 92-branch opcode dispatcher that also lazily triggers Findings 6/7, installs the process-notify callback, and gates the entire injection/token-theft toolkit (Finding 8)** — Confidence: High (registration and full dispatcher structure now confirmed via direct disassembly; see Finding 3a). (*T1112 – Modify Registry, and potentially T1547 if used for persistence-hiding*)
6. **Manual module/API resolution evading standard hooking/monitoring** — Confidence: High. (*T1106 – Native API*)
7. **Kernel-mode win32k GUI subversion** (cross-session window enumeration, title/class harvesting, synthetic message posting) — Confidence: High for capability presence; Medium for operational use, since not all call sites were traced to completion. (*T1010 – Application Window Discovery; T1056 – possible Input Capture / UI injection via `NtUserPostMessage`*)
8. **Keyboard/mouse class-driver reconnaissance** (`kbdclass`, `mouclass`, `kbdhid`, `mouhid`, `i8042prt`) — Confidence: High for targeting/reconnaissance; Low/Unconfirmed for an active capture mechanism — no IRP-attach import (`IoAttachDeviceToDeviceStack`, `IoofCallDriver`) exists in this binary. (*T1056.001 – Input Capture: Keylogging, tentative*)

Capability Flow Diagram



Capability flow diagram: DriverEntry branches into self-delete, module resolution, registry callback, and object callback; the registry callback's opcode dispatcher (Finding 3a) further branches into 8 case-gated capabilities

Diagram reflects only call/reference relationships confirmed during this analysis (see per-finding Confidence ratings above); dotted areas of behavior — e.g. exactly how the four protected PIDs are populated, or how kbdclass reconnaissance results are consumed — are inferred, not directly observed executing. Findings 3, 5's precursor, 6, 7, and 8 were previously diagrammed as independent branches off DriverEntry (or, for Finding 8, listed only as unconfirmed imports); direct disassembly of fcn.140006254 (Finding 3a) confirmed that the process-notify registration, keyboard/mouse reconnaissance, win32k table scan, and the entire injection/token-theft toolkit are in fact opcode-gated branches fired from inside the registry callback itself, not separate DriverEntry-time installs.

Technical Analysis

1. Anti-forensics: self-deletion while loaded

T1070.004 – Indicator Removal: File Deletion

DriverEntry calls into fcn.140005e0c, which opens its own backing .sys file, unlinks its in-memory section mapping, and deletes the file — all while the driver's code remains resident and running in kernel memory. Annotated disassembly of the decision path (radare2, ntoskrnl.exe import names resolved via relocation):

```
; fcn.140005e0c – self-delete routine, called once from DriverEntry
; -----
; Purpose: delete this driver's own .sys file from disk while it is
; still loaded and executing, so no on-disk artifact survives reboot
; or forensic imaging. Classic anti-forensic technique for kernel
; rootkits ("self-deleting driver").
; -----

0x140005eae    call sym.imp.ntoskrnl.exe_IoCreateFileEx
               ; Opens the driver's own backing file by path (path
               ; string supplied by the caller in DriverEntry – the
               ; driver already knows its own registry/service path).
0x140005eb6    js  0x140005f47                ; jump out on failure (NTSTATUS < 0)

0x140005ebc    mov r8, qword [IoFileObjectType]
0x140005ec7    call sym.imp.ntoskrnl.exe_ObReferenceObjectByHandleWithTag
               ; Converts the returned HANDLE into a referenced
               ; FILE_OBJECT pointer so the driver can touch its
               ; internal fields directly (bypassing the file-handle API
               ; surface that AV minifilters typically monitor).
0x140005ef5    jns 0x140005f03                ; success path

0x140005f03    mov rax, qword [arg_6fh]          ; rax = FILE_OBJECT*
0x140005f07    mov rcx, qword [rax + 0x28]        ; rcx = FILE_OBJECT->SectionObjectPointer
0x140005f0b    mov qword [rcx + 0x10], 0
               ; Zeroes SectionObjectPointer->ImageSectionObject –
               ; detaches the cached memory-mapped image section from
               ; the file object, which is the prerequisite step that
               ; Lets ZwDeleteFile succeed on the still-mapped image.
0x140005f13    call sym.imp.ntoskrnl.exe_MmFlushImageSection
               ; Flushes/invalidates the image section so the memory
               ; manager releases its hold on the backing file.
0x140005f1f    call sym.imp.ntoskrnl.exe_ObfDereferenceObject
0x140005f2b    call sym.imp.ntoskrnl.exe_ObCloseHandle
0x140005f33    jz  0x140005f45                ; b1 == 0 (MmFlushImageSection failed) -> skip delete

0x140005f39    call sym.imp.ntoskrnl.exe_ZwDeleteFile
               ; The actual on-disk delete. Executes successfully here
               ; specifically because the image section was unmapped
               ; above – this ordering is what makes self-deletion of a
               ; running driver possible at all.
0x140005f47    ; return NTSTATUS (edi) to DriverEntry
```

Confidence: High — full call chain and API sequence directly disassembled and verified.

2. Manual module resolution

T1106 – Native API

fcn.140006114 calls `ZwQuerySystemInformation(SystemModuleInformation)` twice (a size-probe pass followed by a fill pass), then walks the returned module list performing case-insensitive name comparisons against caller-supplied names (including `ntoskrnl.exe/ntkrnlpa.exe`). This resolves a module's base address without touching `PsLoadedModuleList` directly or calling documented, more heavily monitored resolution APIs.

Locating `win32kfull.sys` and pattern-scanning it for `W32pServiceTable` is handled by a **separate, distinct function chain** (fcn.140001218 and its sub-calls) — see Finding 6 for the full annotated disassembly; that chain reuses only fcn.1400010f8, a structurally similar but independent `PsLoadedModuleList` walker, not fcn.140006114 itself.

Confidence: High.

3. Registry filtering

T1112 – Modify Registry

`CmRegisterCallbackEx` is registered in `DriverEntry` with the altitude string "429628", a valid UTF-16LE string, independently confirmed to be present in the binary. This installs a system-wide registry mini-filter capable of inspecting, blocking, or falsifying any registry read/write/delete/enumerate operation on the host. The registered notify-function pointer is 0x140006254 (fcn.140006254) — see Finding 3a for its full internal structure, now confirmed by direct disassembly.

Confidence: High for registration and for the notify routine's dispatcher structure (see 3a) — individual opcode branches beyond those documented below were not all traced to full behavioral completion.

3a. Registry callback is a unified 92-branch opcode dispatcher — and the true source of Findings 6/7's triggers

T1112 – Modify Registry; ties together T1010, T1056.001, T1547

Disassembly of 0x140006254 (registered as the `CmRegisterCallbackEx` notify routine at entry0 @ 0x14000606d) shows it is **one function**, not several independent init-time calls: 764 instructions, 167 basic blocks, cyclomatic complexity 92, spanning 0x140006254–0x14000703c. It extracts an opcode/selector value and dispatches through a two-tier cascading comparison chain (`sub rax, N; je ...`, repeated for roughly 20 distinct case values), each case invoking a different downstream handler:


```

; fcn.140006254 - CmRegisterCallbackEx notify routine (registry filter callback)
; registered at entry0 @ 0x14000606d, altitude "429628"
; -----
0x140006254    push rbx/rsi/rdi/r12/r13...    ; standard prologue
...
; first dispatch tier - opcode values 0..9
0x1400069c3    sub rax, 0xa                ; case 10
0x1400069d1    je 0x140006af1
0x1400069db    je 0x140006adc
0x1400069e5    je 0x140006acb
0x1400069ef    je 0x140006a80
0x1400069f9    je 0x140006a3f
0x1400069ff    je 0x140006a17
0x140006a05    cmp rax, 1; jnz 0x140006e84    ; default/error path -> STATUS_UNSUCCESSFUL

; second dispatch tier - opcode values 10..28 (continuation block, fcn.140006bc2)
0x140006bca    sub rax, 0x1b                ; case 27
0x140006bce    ja 0x140006e46
0x140006bd4    je 0x140006e26                ; case 27
0x140006bda    sub rax, 0x13; je 0x140006e06    ; case 19
0x140006be4    je 0x140006dc0                ; case 3
0x140006bee    je 0x140006db4                ; case 2
0x140006bf8    je 0x140006d0d                ; case 1
0x140006c02    je 0x140006ceb
0x140006c0c    je 0x140006cda
0x140006c1a    je 0x140006c4d

```

Several of the branches materially change the picture drawn by other findings in this report:

- 0x140006d0d (case 1):** compares the current process's PID (IoGetCurrentProcess → PsGetProcessId) against a cached global (0x140012b50), and on first hit caches the calling process as a tracked target (0x140012b48). **Note:** these globals (0x140012b50/0x140012b48) are distinct addresses from Finding 4's four AccessMask-gating globals (0x140012b80/0x140012b88/0x140012b90/0x140012b98) — no data/control-flow link between case 1's tracked target and Finding 4's protected-PID mechanism was directly traced in this pass. Case 1 is a plausible candidate for a process-tracking role that feeds into Finding 4 indirectly, but this is unverified and not confirmed (see Assessment Limitations).
- 0x140006e91 (reached via case 28's chain):** calls sym.imp.ntoskrnl.exe_PsSetCreateProcessNotifyRoutineEx directly. This confirms the process-creation notify routine (previously listed only as an "added finding" in the audit appendix, with its trigger point undocumented) is installed **on a specific registry-callback opcode path**, not unconditionally at DriverEntry.
- 0x140006ee3 / 0x140006f61:** check the keyboard (0x140013048/0x140013058) and mouse (0x140013040/0x140013050) address-range globals from Finding 7, and call fcn.14000be44 (keyboard recon) / fcn.14000bf94 (mouse recon) **only if those globals are still zero** — i.e., keyboard/mouse class-driver reconnaissance is lazily triggered from inside the registry callback on its first relevant invocation, not run unconditionally during driver load.

- **Case 16 (0x140006a17):** calls `fcn.140001218` directly — the top-level driver of the win32k `W32pServiceTable` masked-pattern-scan chain documented in Finding 6. This confirms that resolver, too, is opcode-gated from inside the registry callback rather than running unconditionally once at `DriverEntry`.
- **Cases 12/13/14 (0x140006adc/0x140006acb/0x140006a80):** route into the full process-injection and token-theft toolkit documented in Finding 8 — remote thread injection (`fcn.14000741c`), chained `LoadLibraryA` DLL injection (`fcn.140008148`), and an 8-way process-sabotage fan-out including a primary-token content-swap primitive (`fcn.1400049b4/fcn.140005bdc`). These are the most severe capabilities in the entire binary, and like the branches above, they are dormant until a matching registry operation reaches this dispatcher.

The function's exit path (0x140007007–0x14000701f) is wrapped in structured exception handling: on an exception, it calls `DbgPrint` with the literal string "[Driver] RegistryCallback: exception caught\n" (0x14000d250) before returning cleanly — direct, unambiguous confirmation that this entire function is the registry-filter callback body, and that its developer explicitly anticipated and guarded against faults in this code path.

Practical implication: Findings 3, 4, 6, 7, and 8 are not independent capabilities wired up once at driver load — findings 4 (partially), 6, 7, and 8 (entirely) are conditionally executed *from inside* the registry callback, gated on which registry operation triggers it. A live system where this driver is loaded but no matching registry operation has yet occurred may show none of the keyboard/mouse reconnaissance, process-notify registration, the win32k table scan, or the injection/token-theft toolkit as having run at all — despite all of them being reachable the moment a correctly crafted registry operation arrives.

Unresolved: the opcode/selector-extraction logic itself. Every branch documented above and in Finding 8 is reached via a single integer “opcode” value compared against case constants (`sub rax, N; je ...`). Where that integer comes from — which field of which registry operation (key path, value name, value data, operation type, or some combination/hash of them) is decoded into this selector — was **not traced in this pass**. The selector-extraction code runs earlier in `fcn.140006254`, before the first `sub rax, 0xa` at 0x1400069c3, and was not disassembled here. This is the single most consequential open question in the report: without it, there is no way to state what a real-world attacker-supplied (or accidentally-triggering) registry operation would need to look like to reach the RCE/token-theft branches (cases 12/13/14), the win32k resolver (case 16), or the process-notify install (case 28's chain). **Recommended follow-up:** disassemble `fcn.140006254` from its entry (0x140006254) up to 0x1400069c3 and identify (a) which `CM_NOTIFY_FUNCTION` argument fields are read, (b) any hashing/transform applied to derive the selector, and (c) whether the selector space is sparse (a handful of magic values) or dense (any operation on a specific key triggers a case) — this materially changes both detection strategy and exploitability assessment.

Confidence: High — function boundaries, dispatch structure, and all cited branch targets were independently confirmed via direct disassembly (not inferred from symmetry or naming). **Unidentified:** the selector-extraction/opcode-derivation logic itself (see above).

4. Process/thread handle manipulation — EDR-blinding

T1562.001 – Impair Defenses: Disable or Modify Tools

`fcfn.1400095b0` registers an `ObRegisterCallbacks` filter on `PsProcessType` with altitude "166666". The callback at `0x140008a9c` tracks four global PID values and rewrites the requested `AccessMask` on `OB_OPERATION_HANDLE_CREATE/OB_OPERATION_HANDLE_DUPLICATE` operations:

Global	Compared against	Action	Bits affected (verified against <code>PROCESS_ACCESS_MASK</code>)
<code>0x140012b80</code>	Target process PID (the process being opened)	Strip	<code>PROCESS_TERMINATE (0x1)</code> , <code>PROCESS_VM_OPERATION (0x8)</code> , <code>PROCESS_VM_READ (0x10)</code> , <code>PROCESS_VM_WRITE (0x20)</code> , <code>PROCESS_SUSPEND_RESUME (0x800)</code>
<code>0x140012b88</code>	Target process PID	Grant unconditional full access (<code>0x1FFFFFF</code>)	—
<code>0x140012b90</code>	Calling process PID (via <code>PsGetCurrentProcessId</code>)	Grant unconditional full access to whatever it opens	—
<code>0x140012b98</code>	Calling process PID	Strip (same bit set as row 1)	Same as row 1

Net effect: a designated target process becomes unkillable, unreadable, and un-suspendable by any other process on the system — including antivirus, EDR agents, and Task Manager — while a separate, cooperating controller process is granted unrestricted access to anything it opens.

Registration site (`fcfn.1400095b0`) — sets up the callback and its altitude:

```
; fcn.1400095b0 - installs the ObRegisterCallbacks process filter
; -----
; Purpose: register a kernel callback that intercepts every handle
; open/duplicate against PsProcessType (i.e. every OpenProcess-style
; operation system-wide) and rewrites the granted access mask before
; the caller ever sees the handle.
; -----
```

```

0x1400095eb    lea rdx, ["166666"]           ; altitude string for this filter
0x1400095f6    call sym.imp.ntoskrnl.exe_RtlInitUnicodeString
0x1400095fc    mov rax, qword [PsProcessType] ; object type to filter: processes
0x140009610    lea rax, [0x140008a9c]       ; -> pre-operation callback function
0x140009626    lea rdx, [0x140012b78]       ; RegistrationContext / altitude block
0x140009632    call sym.imp.ntoskrnl.exe_ObRegisterCallbacks
                ; From this point on, 0x140008a9c fires synchronously on
                ; every process handle create/duplicate on the system.

```

Callback body (0x140008a9c) — the access-mask rewrite logic, keyed off arg2->ObjectType/PID comparisons (rbx = arg2; dword [rbx] = operation type; qword [rbx+0x20] = pointer to the in/out AccessMask structure):

```

; fcn.140008a9c - OB_PREOP_CALLBACK, fires on every process handle op
; -----
; arg2 (rdx) = POB_PRE_OPERATION_INFORMATION*
;   dword [rbx] = Operation (1 = HANDLE_CREATE, 2 = HANDLE_DUPLICATE)
;   qword [rbx+0x20] = ptr to the CREATE/DUPLICATE parameter block,
;                     whose DWORD[0]/[4] are the DesiredAccess /
;                     OriginalDesiredAccess masks this code clears
;                     bits from in place.
; -----

0x140008aa9    call sym.imp.ntoskrnl.exe_PsGetProcessId
                ; rax = PID of the process TARGET of this open (the
                ; process being opened, not the caller).

0x140008aaf    cmp rax, qword [0x140012b80] ; target PID == "protected" PID?
0x140008abf    jne 0x140008b81             ; no -> check the "whitelisted" PID instead

; --- protected-process path: strip dangerous bits from the mask ---
0x140008ae2    mov eax, dword [rcx]           ; eax = current AccessMask
0x140008ae4    test al, 1                     ; PROCESS_TERMINATE (0x1)?
0x140008ae8    and eax, 0xffffffff           ; clear it
0x140008af3    test al, 8                     ; PROCESS_VM_OPERATION (0x8)?
0x140008af7    and eax, 0xffffffff7             ; clear it
0x140008b02    test al, 0x10                    ; PROCESS_VM_READ (0x10)?
0x140008b06    and eax, 0xfffffffffef          ; clear it
0x140008b11    test al, 0x20                    ; PROCESS_VM_WRITE (0x20)?
0x140008b15    and eax, r10d                  ; r10d = 0xffffffffdf -> clear it
0x140008b20    bt eax, 0xb                      ; PROCESS_SUSPEND_RESUME (bit 11, 0x800)?
0x140008b26    btr eax, 0xb                      ; clear it
                ; ^ identical strip sequence is repeated once more at
                ; 0x140008b2c for HANDLE_DUPLICATE (dword[rbx]==2),
                ; and a third time at 0x140008bef guarded by the
                ; "calling process == 0x140012b98" check further down.

; --- whitelisted-target path: unconditional full access ---
0x140008b81    cmp r8, qword [0x140012b88] ; target PID == "whitelisted" PID?
0x140008b9e    mov dword [rax], 0x1fffffff ; grant PROCESS_ALL_ACCESS
0x140008ba8    mov dword [rax+4], 0x1fffffff ; (both mask fields)

; --- trusted-caller path: whoever IS this driver's controller gets ---
; --- full access to whatever process it opens ---
0x140008baf    call sym.imp.ntoskrnl.exe_PsGetCurrentProcessId
0x140008bb5    mov rcx, qword [0x140012b90] ; "controller" PID
0x140008bc7    cmp rax, rcx                     ; caller PID == controller PID?
0x140008bcd    mov dword [rcx], 0x1fffffff ; grant PROCESS_ALL_ACCESS

```

Confidence: High — callback logic, altitude string, and all four global roles directly disassembled and the access-mask bit values verified against documented Windows `PROCESS_ACCESS_MASK` constants.

Additionally, `PsSetCreateProcessNotifyRoutineEx` is invoked from `0x140006e91`, an opcode-gated branch of the registry-callback dispatcher documented in Finding 3a — its call site is confirmed. However, the registered process-creation callback's body (likely its role: populating the tracked-PID globals above) was not traced in this pass.

5. Direct CR3 read used for physical-to-virtual address resolution

T1014 – Rootkit (narrow scope — see correction below)

A follow-up disassembly pass, triggered by an external analysis claim (see External Intelligence Corroboration, below), located a genuine `mov rcx, cr3` instruction at `0x140008dd6`, inside `fcn.140008db8`. An exhaustive opcode sweep for all `mov reg, cr3` encodings (`0f20d8–0f20df`) across the entire binary confirms this is the **only** such instruction present — there is exactly one CR3 access in the whole driver, not a general page-table-patching routine.

```
; fcn.140008db8 – physical-address-to-virtual-address resolver
; -----
; Purpose: given a physical address (arg2/rdx), find the virtual
; address that currently maps it, by walking the list of physical
; memory ranges Windows reports via MmGetPhysicalMemoryRangesEx and
; testing candidate PFN-derived addresses with MmIsAddressValid.
; -----

0x140008dd6  mov rcx, cr3
            ; Reads the current page-table base (PML4 physical
            ; address) into rcx. This is the only genuine.
            ; CR3 access anywhere in this 74KB binary.
0x140008dd9  call sym.imp.ntoskrnl.exe_MmGetVirtualForPhysical
            ; NOTE: rcx (cr3) is NOT actually consumed as this
            ; call's argument in the traced path – MmGetVirtualForPhysical
            ; is a *documented* kernel API that itself performs the
            ; official physical->virtual translation. The adjacent
            ; CR3 read appears to be an independent, redundant
            ; capture of the page-table base rather than a manual
            ; page-table-entry walk substituting for this API.
0x140008df3  call sym.imp.ntoskrnl.exe_MmGetPhysicalMemoryRangesEx
            ; Documented API: returns the list of physical address
            ; ranges backed by real RAM.
0x140008e0a  shr r14, 9                ; r14 = candidate PFN >> 9
0x140008e18  and r14, 0x7fffffff8
0x140008e1b  add r14, rbx              ; index into the ranges list
0x140008e79  ...
0x140008e8f  call sym.imp.ntoskrnl.exe_MmIsAddressValid
            ; Validates each candidate address before dereferencing it
            ; – defensive probing typical of manual pointer-chasing
            ; through the memory-range list, not page-table-entry
            ; bit manipulation.
0x140008ef6  call sym.imp.ntoskrnl.exe_MmIsAddressValid
0x140008f2d  call sym.imp.ntoskrnl.exe_ExFreePoolWithTag
            ; Frees the pool buffer returned by
            ; MmGetPhysicalMemoryRangesEx once the walk completes.
```

Corrected assessment: the earlier working conclusion in this investigation (“no CR3 access, VT overreached”) was wrong — this exhaustive pass found the real call site. However, the previously reported description of this finding also overstated it: **no `invlpg` instruction exists anywhere in this binary** (verified by opcode search), and the surrounding code does not write to or manually walk raw page-table-entry bits — it walks a documented

`MmGetPhysicalMemoryRangesEx` range list using `MmIsAddressValid` guards, calling the documented `MmGetVirtualForPhysical` API alongside a CR3 read whose output is not visibly consumed downstream. This is genuine, unusual, low-level memory introspection — but it is address *resolution/reconnaissance*, not confirmed page-table *write-protection bypass or patching*. The separate `mov rax, cr3` at `0x1400084ea` (noted in an earlier pass) remains an IRQL save under `cli`, unrelated to paging.

Confidence: High for the CR3 instruction’s existence and location; **Medium-Low** for any claim that it performs page-table write-protection bypass — the traced code is consistent with physical-memory range enumeration, not PTE patching.

6. Kernel-mode GUI/window subversion

T1010 – Application Window Discovery; possible T1056 – Input Capture (UI-level)

FLOSS deobfuscation recovered win32k syscall names that are absent from plaintext strings output (dynamically constructed/decoded at runtime): `NtUserFindWindowEx`, `NtUserQueryWindow`, `NtUserGetForegroundWindow`, `NtUserBuildHwndList`, `NtUserInternalGetWindowText`, `NtUserGetClassName`, `NtUserGetWindowDC`, `NtUserGetDCEx`, `NtUserPostMessage`, plus the literal string `w32pServiceTable`.

From kernel mode — invisible to the user-mode API hooking most AV/EDR products rely on — this capability set allows the driver to: - Enumerate all windows across **all user sessions**, not just the current one - Read window titles and class names (surveillance of what is open/running) - Obtain device contexts for arbitrary windows (screen-scraping without user-mode `GetDC`) - Post synthetic messages to arbitrary windows via `NtUserPostMessage` (UI automation / potential UIPI bypass against elevated or protected dialogs)

Confidence: High for capability presence (strings and the `w32pServiceTable` signature-scan target are unambiguous); **Medium for operational scope**, since not every call site consuming these resolved syscalls was traced end-to-end.

Annotated disassembly of the `w32pServiceTable` pattern-scan chain — this is a distinct code path from the generic module resolver in Finding 2 (`fcn.140006114`, which resolves `ntoskrnl.exe/ntkrnlpa.exe` by walking `PoLoadedModuleList/ZwQuerySystemInformation`). The win32k table is instead located via a classic IDA/x64dbg-style masked byte-signature scan, four functions deep:

```
; fcn.140001218 – top-level "find w32pServiceTable" driver
; -----
; Supplies the four scan parameters and stores the resolved table
; pointer into a global.
; -----
```



```

0x140001231 lea rcx, ["win32kfull.sys"] ; target module name (0x14000cd30)
0x14000122a lea rdx, [".text"] ; target section name (0x14000cd20)
0x140001223 lea r8, [pattern_bytes] ; 0x14000cd10: e8 00 00 00 00 8b f8 85 c0 75 0e
0x14000121c lea r9, ["x????xxxxxx"] ; mask (0x14000cd00): x=exact match, ?=wildcard
0x140001238 call fcn.140001090 ; -> resolves match address in rax
0x14000123d mov rdx, rax
0x140001243 test rax, rax
jz fail -> return STATUS 0xc0000141
0x14000124c eax = dword [rax + 1] ; read the call's rel32 operand (the wildcarded bytes)
0x14000124f eax += 5 ; adjust to end of the 5-byte CALL instruction
0x140001255 rcx = eax + rdx ; resolve rel32 -> absolute target = W32pServiceTable
0x140001258 qword [0x1400126d0] = rcx ; cache the resolved table pointer globally

```

The pattern `e8 00 00 00 00 8b f8 85 c0 75 0e` with mask `x????xxxxxx` matches the instruction sequence `call <rel32>; mov edi, eax; test eax, eax; jnz +0xe` — a compiler-emitted idiom found adjacent to `W32pServiceTable`'s initialization inside `win32kfull.sys`. The four wildcarded bytes are the `call`'s relative displacement, which the code reads back out of the matched instruction stream and resolves to an absolute address — this is how the driver obtains the internal, non-exported table pointer without any public symbol or export.

```

; fcn.140001090 - resolve module base, locate its .text section, scan it
; -----
; arg1=module name, arg2=section name, arg3=pattern bytes, arg4=mask
; -----
0x1400010b6 call fcn.1400010f8 ; walk PsLoadedModuleList, RtlCompareUnicodeString
; against arg1 ("win32kfull.sys") -> module base
0x1400010c0 rbp = qword [rax + 0x30] ; LDR_DATA_TABLE_ENTRY.DLLBase
0x1400010ca call fcn.140001170 ; walk the PE section table for arg2 (".text"),
; case-insensitive 8-byte name compare -> section VA
0x1400010e3 call fcn.140001030 ; byte-for-byte masked scan of that section
; for arg3/arg4 (pattern/mask) -> match address

; fcn.140001030 - masked byte-pattern scanner (the actual "signature scan")
; -----
0x14000105e cmp byte [r8], 'x' ; mask byte: 'x' = must match exactly
0x140001062 jnz skip_check
0x140001064 al = byte [rdx] ; pattern byte
0x140001066 cmp byte [r9], al ; compare against scanned module byte
0x140001069 jnz mismatch -> advance outer offset, retry
0x14000106b r8++; r9++; rdx++ ; '?' mask bytes fall through unchecked (wildcard)
0x140001074 cmp byte [r8], 0 ; mask terminator reached -> full pattern matched
0x140001078 jnz loc_0x14000105e

```

This is the standalone masked-scan primitive: for each candidate offset in the target section, it walks the mask string byte-by-byte — 'x' bytes force an exact comparison against the corresponding pattern byte, any other mask byte (here '?') is skipped unconditionally — and returns the first offset where the whole mask is satisfied.

Confidence: High — the full four-function chain (name resolution → section lookup → masked scan → rel32 fixup) was independently disassembled and is internally consistent with the recovered `"win32kfull.sys" / ".text" / pattern/mask` constants; this materially sharpens the earlier “signature scan” description in Finding 2/6 by showing it is a generic, reusable primitive rather than ad hoc code, and that it is entirely distinct from the

`PsLoadedModuleList`-walk resolver documented in Finding 2 (that resolver is reused here only as the first stage, via `fcn.1400010f8`).

7. Keyboard/mouse class-driver reconnaissance

T1056.001 – Input Capture: Keylogging (tentative — see caveat)

Two structurally identical functions were disassembled:

- fcn.14000be44 (keyboard path): resolves `\Driver\kbdhid` or `\Driver\i8042prt` (HID or legacy PS/2 stack, whichever is present) via `ObReferenceObjectByName(..., IoDriverObjectType, ...)`, then separately resolves `\Driver\kbdclass` the same way, then walks that driver object's `DeviceObject` linked list via helper fcn.14000c0e4.
- fcn.14000bf94 (mouse path): identical structure, targeting `\Driver\mouhid`/`\Driver\i8042prt` then `\Driver\mouclass`, walking its `DeviceObject` linked list via a **separate but structurally identical helper**, fcn.14000c208 (not a shared/reused function — each input class has its own copy of the walk logic).

Both list-walk helpers were independently disassembled and confirmed structurally identical: each re-validates the driver object's name via `RtlCompareUnicodeString`, then walks a secondary linked list at `[device+0x40]`, using `MmIsAddressValid` to compute and record a **minimum/maximum valid-address range** into a dedicated pair of globals — `0x140013040/0x140013050` for the mouse path (fcn.14000c208), `0x140013048/0x140013058` for the keyboard path (fcn.14000c0e4).

Annotated disassembly of the mouse-side helper, fcn.14000c208 (called from fcn.14000bf94 at `0x14000c0ab`):

```
; fcn.14000c208 - mouclass device-object list-walk / address-range recon
; (mouse-side twin of fcn.14000c0e4, which performs the same role for
; the keyboard/kbdclass path)
; -----
; arg1 (rcx) = driver object context block (from ObReferenceObjectByName
;           on "\Driver\mouhid"/"\Driver\i8042prt", passed through
;           from fcn.14000bf94)
; arg2 (rdx) = the \Driver\mouclass driver object just resolved
; -----

0x14000c224    lea rdx, ["\Driver\mouclass"]
0x14000c240    call sym.imp.ntoskrnl.exe_RtlInitUnicodeString
               ; Re-materializes the "\Driver\mouclass" name string for
               ; a second identity check below.

; --- walk arg1's List, comparing each entry's name to "mouclass" ---
0x14000c254    mov rcx, qword [rax + 8]
0x14000c258    add rcx, 0x38
0x14000c25c    call sym.imp.ntoskrnl.exe_RtlCompareUnicodeString
               ; Re-validates that the driver object found really is
               ; named "\Driver\mouclass" - defense against a stale or
               ; mismatched pointer before trusting its internal fields.

; --- walk the DEVICE_OBJECT chain hanging off the driver object ---
0x14000c28f    mov r14, qword [rbx + 0x40]      ; r14 = list head at [device+0x40]
0x14000c296    mov qword [0x140013040], 0      ; reset "min valid addr" global
0x14000c2a1    mov rcx, r14
0x14000c2a4    call sym.imp.ntoskrnl.exe_MmIsAddressValid
               ; Probes each List-entry pointer before dereferencing it.
```



```

0x14000c2c2    mov rdi, qword [r14]           ; rdi = candidate pointer
0x14000c2ca    mov qword [0x140013040], rsi  ; record as new minimum
0x14000c2ee    mov qword [0x140013050], rdi  ; record as new maximum
0x14000c2f7    inc r15d                     ; loop counter
0x14000c2fa    inc r14                      ; advance list pointer
0x14000c304    jb 0x14000c2a1               ; loop up to 0x1000 (4096) iterations

; --- outer loop: continue to the next device object in the chain ---
0x14000c30a    mov rax, qword [rsi + 0x10]   ; DEVICE_OBJECT->NextDevice-style link
0x14000c314    jnz 0x14000c289               ; continue until null

0x14000c31e    ; return 0/1 in eax to caller

```

Critical architectural limitation: IoAttachDeviceToDeviceStack, IoCallDriver, and IoCompleteRequest are **not imported anywhere in this binary**. Classic IRP-filter-driver keylogging — attaching a filter device to the stack and intercepting IRP_MJ_READ completion to harvest scan codes — is architecturally absent from this compiled sample. Neither fcn.14000c0e4 nor fcn.14000c208 contains any such call.

Assessment: the driver actively locates and fingerprints the keyboard and mouse class-driver device objects and computes an address range from their internal structures — genuine reconnaissance, plausibly a precursor to reading a buffer or structure directly at a computed offset (a technique some kernel-mode input-capture tools use in place of IRP filter attachment). The code that actually reads keystrokes/mouse events was not found in this pass for either input class.

Confidence: High for driver targeting and enumeration (both keyboard and mouse paths now independently confirmed via direct disassembly of their respective list-walk helpers); **Low/Unconfirmed** for a complete, active input-capture capability in this specific binary as compiled.

8. Process injection / token manipulation toolkit — fully traced to three opcode-gated primitives inside the registry-callback dispatcher

T1055 – Process Injection; T1134 – Access Token Manipulation

All call sites for this toolkit were located and disassembled. They are **not independent, freestanding routines** — all three are additional case targets in the same first-tier opcode dispatch chain documented in Finding 3a (fcn.140006254, reached via its

fcn.140006114-continuation logic), reusing the same sub rax, N; je ... cascade at 0x1400069c3—0x140006a05. Counting cases back from the already-documented case 10 branch:

```

0x1400069c3    sub rax, 0xa                  ; case 10 -> 0x140006b50 (unrelated string op)
0x1400069d1    je 0x140006af1                ; case 11 -> string-based sub-op (not injection-related)
0x1400069db    je 0x140006adc                ; case 12 -> THREAD INJECTION (fcn.14000741c)
0x1400069e5    je 0x140006acb                ; case 13 -> DLL INJECTION (fcn.140008148)
0x1400069ef    je 0x140006a80                ; case 14 -> PID lookup -> 8-way process-sabotage
                                ; fan-out incl. TOKEN SWAP (fcn.1400049b4)
0x1400069f9    je 0x140006a3f                ; case 15 -> (unrelated, string/format helper)
0x1400069ff    je 0x140006a17                ; case 16 -> invokes fcn.140001218, the win32k
                                ; W32pServiceTable scan (Finding 6) -

```

```

;           confirms that resolver is ALSO
;           opcode-gated from this dispatcher,
;           not just run once at DriverEntry

```

Case 12 — remote thread injection (fcn.14000741c, called from dispatcher case 12 at 0x140006ae7):

```

; fcn.14000741c(target_pid, start_addr, start_param) - CreateRemoteThread equivalent
; -----
0x14000749e  call sym.imp.ntoskrnl.exe_PsLookupProcessByProcessId ; resolve target EPROCESS
0x1400074b4  call sym.imp.ntoskrnl.exe_ObfDereferenceObject
0x1400074da  call sym.imp.ntoskrnl.exe_KeStackAttachProcess      ; attach to target's address
space
0x140007517  call sym.imp.ntoskrnl.exe_RtlCreateUserThread        ; create thread at arg3 in
target
0x140007540  call sym.imp.ntoskrnl.exe_ZwWaitForSingleObject      ; (conditionally) wait on it
0x140007553  call sym.imp.ntoskrnl.exe_ZwClose
0x140007563  call sym.imp.ntoskrnl.exe_KeUnstackDetachProcess     ; detach
0x140007576  call sym.imp.ntoskrnl.exe_ObfDereferenceObject

```

A textbook kernel-mode remote-thread-injection primitive: attach to an arbitrary process's address space by PID, start a user-mode thread at a caller-supplied address, then detach. The caller supplies the target PID and start address directly (arg1/arg3 from the dispatcher), so the actual payload address is written elsewhere (not traced in this pass — see caveat below).

Case 13 — LoadLibraryA-based DLL injection (fcn.140008148, called from dispatcher case 13 at 0x140006ad2):

```

; fcn.140008148(target_pid, dll_name_ansi) - LoadLibrary-injection loader
; -----
0x140008171  call sub.ntoskrnl.exe_strlen                        ; length of supplied DLL name string
0x140008189  call fcn.140008950                                ; (name validation/lookup helper)
0x1400081b0  call fcn.1400091c8                                ; (further lookup, unidentified role)
0x140008202  call sym.imp.ntoskrnl.exe_RtlAnsiStringToUnicodeString
0x140008228  call sym.imp.ntoskrnl.exe_RtlFreeUnicodeString
0x140008249  call fcn.140007d28                                ; resolves "LoadLibraryA"'s address
;           ; inside Kernel32.dll (name string at
;           ; 0x14000d2e0) for the TARGET process
0x140008262  call fcn.14000741c                                ; -> hands off to the thread-injection
;           ; primitive above: rcx=target_pid,
;           ; r8=resolved LoadLibraryA address

```

This chains directly into Case 12's thread-injection primitive: it resolves LoadLibraryA's address inside the target process (a Kernel32.dll string at 0x14000d2e0 is used as the module/proc name lookup key), then calls fcn.14000741c(target_pid, dll_name_string, LoadLibraryA_address) — the classic CreateRemoteThread(..., LoadLibraryA, dllPath) **DLL-injection pattern**, implemented from kernel mode.

Case 14 — 8-way process-sabotage fan-out including token theft (fcn.1400049b4, called from dispatcher case 14 at 0x140006ab1):

```

; fcn.1400049b4(pid1, pid2) - validate target still alive, then fan out into 8 subroutines
; -----
0x1400049d1  call sym.imp.ntoskrnl.exe_PsLookupProcessByProcessId

```

```

0x1400049e7  call sym.imp.ntoskrnl.exe_PsGetProcessExitStatus      ; skip if not STATUS_PENDING
                                                    ; (i.e. target must be a LIVE
process)
0x140004a09  call fcn.140004efc      ; unidentified (no notable imports)
0x140004a16  call fcn.140004e18      ; ExAllocatePoolWithTag / ExFreePoolWithTag (buffer helper)
0x140004a23  call fcn.140004bc0      ; MmIsAddressValid, ExAllocatePool (address-probe helper)
0x140004a30  call fcn.140004d0c      ; RtlGetVersion-gated (OS-version-dependent code path),
                        ; MmIsAddressValid, ExAllocatePool
0x140004a3d  call fcn.140005bdc      ; ** TOKEN SWAP ** (see below)
0x140004a4a  call fcn.14000566c      ; KeStackAttachProcess -> ZwAllocateVirtualMemory -> ...
                        ; -> KeUnstackDetachProcess (shellcode-staging pattern)
0x140004a57  call fcn.1400053f8      ; identical KeStackAttachProcess/ZwAllocateVirtualMemory
                        ; /KeUnstackDetachProcess pattern
0x140004a5f  call fcn.1400050f8      ; identical pattern again
0x140004a67  call fcn.140004f40      ; identical pattern again

```

Four of these eight (fcn.14000566c, fcn.1400053f8, fcn.1400050f8, fcn.140004f40) are structurally near-identical: each attaches to the target process's address space with `KeStackAttachProcess`, calls `ZwAllocateVirtualMemory` to carve out a region in the target, then detaches with `KeUnstackDetachProcess` — consistent with **manual shellcode staging** (allocate + write memory in a remote process without necessarily creating a thread there, e.g. to stage a payload the case-12 thread-injection primitive later executes). The exact write/copy step into the allocated region was not isolated to a specific instruction in this pass.

Token-swap primitive (fcn.140005bdc):

```

; fcn.140005bdc(dest_process, src_process) - primary-token content swap
; -----
0x140005bf1  call sym.imp.ntoskrnl.exe_PsReferencePrimaryToken ; on arg1 (dest)
0x140005bfd  call sym.imp.ntoskrnl.exe_PsReferencePrimaryToken ; on arg2 (src)
0x140005c09  call fcn.140004b68      ; resolve token-internal structure pointer (dest)
0x140005c14  call fcn.140004b68      ; resolve token-internal structure pointer (src)
0x140005c23  xmm0 = xmmword [rax]    ; read 16 bytes from src token structure
0x140005c26  xmmword [rbx] = xmm0    ; write into dest token structure
0x140005c29  xmm1 = xmmword [rax + 0x10] ; read next 16 bytes
0x140005c2d  xmmword [rbx + 0x10] = xmm1 ; write into dest token structure
0x140005c34  call sym.imp.ntoskrnl.exe_ObfDereferenceObject ; release src token
0x140005c4c  goto sym.imp.ntoskrnl.exe_ObfDereferenceObject ; (tail-call) release dest token

```

A raw 32-byte in-place copy from one process's primary-token internal structure into another's, via fcn.140004b68 (a token-field resolver whose exact offset semantics were not further decoded). This is a **token-theft/impersonation primitive**: overwriting a chosen process's token fields with another process's token fields grants it that other process's security context (e.g., escalating a low-privilege attacker-controlled process to SYSTEM by copying SYSTEM's token fields into it), entirely from kernel mode and without any documented token-duplication API.

Practical implication: none of Finding 8's capabilities run automatically at driver load. Like Finding 3a's other branches, they are dormant until the registry callback (fcn.140006254) receives a specific registry operation whose extracted opcode/selectors lands on case 12, 13, or 14 — i.e., this whole toolkit is remotely triggerable via a crafted registry operation matching the driver's expected opcode encoding, not merely present-but-inert code.

Confidence: High for all three call chains (thread injection, DLL injection, token swap) and their dispatcher gating — each was independently disassembled end-to-end. **Medium** for the four `KeStackAttachProcess/ZwAllocateVirtualMemory` “shellcode-staging” helpers under case 14 — their structural similarity and API sequence is confirmed. Still, the exact data written into the allocated region was not isolated to a specific call site in this pass. **Unidentified:** `fcn.140004efc` (no notable imports; role unclear) and the exact opcode/selector encoding an external caller would need to supply to reach cases 12/13/14 (the selector-extraction logic itself lives earlier in `fcn.140006254`, before the `sub rax,N` cascade, and was not re-derived in this pass).

External Intelligence Corroboration

The following was sourced from VirusTotal, MalwareBazaar, and an associated X/Twitter post on 2026-07-06, and reflects **externally reported information, not independently re-verified by this analyst today** except where explicitly marked “matches local finding.”

VirusTotal ([virustotal.com/gui/file/c7ec3...074f6](https://www.virustotal.com/gui/file/c7ec3...074f6)): - 13/70 vendor detections at time of check; community score -58. Notable labels: Microsoft Trojan:Win32/Wacatac.B!ml, ESET-NOD32 Win64/Agent.ITG, GData Win64.Trojan.Agent.SKPQ8W, Rising Trojan.Agent!8.B1E, Avast/AVG MalwareX-gen [Misc]. - VT’s automated AI “Code Insights” summary independently claimed: kernel-mode process/DLL injection, direct kernel memory patching / page-table manipulation via CR3, keyboard/mouse interception targeting `kbdclass/mouclass`, `ObRegisterCallbacks`-based process tampering, and XOR-based string obfuscation. - The kernel-mode process/DLL injection claim **matches local finding** (Finding 8) exactly — remote thread injection and chained `LoadLibraryA` DLL injection were both fully disassembled to specific opcode-gated call sites. - **The CR3 claim is partially corroborated.** A genuine `mov rcx, cr3` was located (see Finding 5), correcting this analyst’s own earlier, mistaken refutation. However, VT’s characterization (“page-table manipulation... to bypass write protection”) is broader than what the code does: the traced routine performs physical-to-virtual address resolution using documented `Mm*` APIs, not a manual page-table-entry patch. No `invlpg` exists in the binary. Treat VT’s specific “bypass write protection” framing as unconfirmed/likely overstated. - The `ObRegisterCallbacks` and process-tampering claim **matches local finding** (Finding 4) exactly. - The keyboard/mouse interception claim **partially matches local finding**: targeting and reconnaissance confirmed (Finding 7); a complete active-capture mechanism was not localized. - No Authenticode signature — **matches local finding**. - Hashes, `ImpHash`, compile timestamp, and section table as reported by VT were consistent with local extraction (not independently re-verified against VT’s raw report today; treat as corroborating, not primary, evidence). - Alternate filenames reported: `hf1ar.exe` and `7.4.1.sys` (the latter independently consistent with the sample’s true nature as a driver). - First VT submission: 2026-07-04 12:15:32 UTC — 34 seconds after the local compile timestamp, suggesting the submitter is close to or is the build source. - A community comment attributed to `thor` (Nextron Systems / THOR APT Scanner) cites a YARA match against `Winnti_ZxShell_Dec17_1` (rule author: Florian Roth, Valhalla private feed). **This is a 2017-era stylistic/technique-pattern rule match, not a confirmed threat-actor attribution.** It indicates code-level similarity to previously catalogued Winnti/ZxShell kernel-rootkit tooling (self-deleting driver, `ObRegisterCallbacks` process

protection, registry filtering, and manual API resolution all overlap), which could reflect genuine lineage, reuse of leaked/public source, or an independently convergent design. No further attribution claim is made here. - The MalwareBazaar submission is credited to JAMESWT_WT, a known public malware-intelligence researcher, plausibly the source of the referenced X/Twitter post.

MalwareBazaar (bazaar.abuse.ch/sample/c7ec3...074f6): - Listed as “Threat unknown” (no family tag assigned), 7 vendor detections. - Multi-engine panel: ReversingLabs Win64.Trojan.Generic, Hybrid Analysis Win64/Agent.ITG, Nucleon Malprob Malware, FileScan.IO Likely Malicious, Spamhaus Suspicious file; Kaspersky OpenTIP Clean, Malva.RE inconclusive — a split consistent with a low-prevalence, freshly-built sample. - Three public YARA hits: CP_Script_Inject_Detector (generic process-injection heuristic, consistent with local findings), FreddyBearDropper, and VECT_Ransomware. The latter two are broad, generic-technique rules; **the ransomware match is treated with skepticism** — no ransom-note strings, mutex-creation patterns, or traced encryption-API call sites were found in local disassembly. - First seen on Bazaar: 2026-07-05 07:15:45 UTC.

Local environment checks are not indicative of anything: ClamAV returned a clean/no-detection result, but the signature database was more than 7 days out of date at scan time — **not treated as a meaningful negative signal**. - capa did not produce output; its embedded default rule set was missing in the analysis environment, and it was not re-run with an external rules repository. This is a **tooling gap**, not evidence of anything about the sample.

Indicators of Compromise

Type	Value
SHA-256	c7ec30029a10580a0b33b71b75d34efe053cc34d908bf75998f457a0c1f074f6
ImpHash	93aaf72477167a05c85c15e6440d1c06
Alternate filenames (externally reported)	hflar.exe, 7.4.1.sys
PDB path	\AppData\Roaming\reasonix\global-workspace\6.15\x64\Release\CrackerDrv.pdb
Debug string	[Driver] RegistryCallback: exception caught
Registry filter altitude	429628
Process (Ob) filter altitude	166666
Compile timestamp	2026-07-04 12:14:58 UTC (PE header; 34 seconds before first VirusTotal submission — see External Intelligence Corroboration)
Non-standard PE section	fothk — 4,096 bytes, 4,091/4,096 bytes are 0xCC filler with a single relative jmp stub at offset +16 targeting 0x14000c5c0;

Type	Value
	consistent with a reserved hook/trampoline scratch region, not compiler-generated (see Sample Identification)
Referenced legitimate system module (target, not itself malicious)	win32kfull.sys
Driver object names referenced	\Driver\kbdhid, \Driver\i8042prt, \Driver\kbdclass, \Driver\mouhid, \Driver\mouclass
Network indicators	None found in static strings or FLOSS output

No on-disk artifact is expected to persist post-infection at the location from which this driver was loaded — the sample deletes its own backing file while resident (Finding 3).

Assessment Limitations

- **Loader/persistence mechanism not present in this sample.** How this driver was placed and loaded on the source machine (test-signing mode, DSE bypass, BYOVD chain, or a signed loader not submitted here) is unknown from this file alone.
- **Configuration/control mechanism for the four protected-PID globals is unconfirmed.** No IOCTL/device interface exists; two unverified candidate mechanisms were identified, but neither was confirmed: case 1 of the registry-callback dispatcher (Finding 3a, 0x140006d0d, distinct globals — no traced link to Finding 4's globals), and the body of the `PsSetCreateProcessNotifyRoutineEx`-registered callback itself (Finding 4's closing note) — its likely role in populating these globals was not traced in this pass.
- **The individual opcode branches in the Registry callback filter logic were not all traced to full behavioral completion.** The dispatcher's overall structure and its highest-severity branches (cases 1, 12, 13, 14, 16, 28 — see Finding 3a) were fully disassembled, but roughly a dozen lower-numbered cases (e.g., the string-handling branches at cases 10, 11, 15) were identified only by target address, not disassembled for behavior; the specific registry keys/values the callback inspects or blocks (as opposed to which opcode branch a given operation reaches) also remain unknown.
- **The opcode/selector-extraction logic that routes a registry operation to a specific dispatcher case was not traced — this is the single most consequential open question in the report (see Finding 3a).** It gates every conditionally executed capability documented here: cases 12/13/14 (Finding 8's injection/token toolkit), case 16 (Finding 6's win32k resolver), and case 28's chain (the process-notify install). The selector-extraction code lives earlier in `fcn.140006254`, upstream of the `sub rax,N` cascade, and was not disassembled in this pass; without it, there is no way to state what a real-world registry operation would need to look like to trigger any of these branches. The four shellcode-staging helpers' exact payload-write step (Finding 8, case 14) was also not isolated to a specific instruction.

- **The actual keystroke/mouse-event capture mechanism, if any, was not found in this binary.** Only driver-object reconnaissance and address-range fingerprinting were confirmed.
- **Not every call site consuming the resolved win32k syscalls (Finding 6) was traced end-to-end.** The `w32pServiceTable` resolution chain itself is fully disassembled and high-confidence, but the full operational scope of how the resolved table pointer is subsequently used was not traced call-by-call.
- **capa did not execute successfully** in this environment (missing embedded rule set) and was not substituted with an external rules repository; ATT&CK technique mappings above are analyst-derived from manual RE, not capa output.
- External intelligence (VT vendor labels, MalwareBazaar YARA hits, Winnti/ZxShell stylistic match) is reported as-is from third parties and was not independently re-derived from raw VT/Bazaar API data during this pass.

Recommendations (for the suspected-infected Windows host)

1. **Check Driver Signature Enforcement / test-signing state:** `bcdedit /enum {current}` — look for `testsSigning Yes` OR `noIntegrityChecks Yes`. Either indicates a deliberate DSE bypass; if found, run `bcdedit /set testsigning off` (and remove any `noIntegrityChecks` override) and reboot before considering the host remediated, since either setting would let a replacement unsigned driver load again immediately.
2. **Check Secure Boot status:** `Confirm-SecureBootUEFI` (PowerShell, admin). If it returns `False`, Secure Boot has been disabled in firmware — re-enable it from the UEFI setup screen (exact menu path is OEM-specific; consult the vendor's firmware documentation) before returning the host to service, since a disabled Secure Boot is itself a plausible loader path for this driver and will remain exploitable otherwise.
3. **Enumerate currently loaded drivers:** `driverquery /v /fo csv > loaded_drivers.csv`, then in the CSV flag any entry whose `Link Date` is within the last 30 days, whose path is outside `C:\Windows\System32\drivers\`, whose `Driver Type` is not `Kernel`, or **whose listed path no longer exists on disk** (Test-Path each entry) — this last check specifically targets Finding 1's self-delete-while-loaded technique, which leaves a driver showing as loaded here with no corresponding backing file (see Detection Opportunities). Any driver flagged by more than one of these four criteria should be treated as a standing candidate for further static analysis, following the same offline/non-executing methodology used for this sample.
4. **Hunt for a BYOVD loader chain:** in the `driverquery /v` output from step 3, check every **signed** driver against Microsoft's [vulnerable driver blocklist](#) (`Get-MpPreference / WDAC` policy on Windows 11 24H2+ enforces this list by default — confirm it is actually enabled with `Get-CimInstance -Namespace root/Microsoft/Windows/CI -ClassName MS_VulnerableDriverBlocklist` or equivalent) — common abused categories are physical-memory-access drivers (e.g. those exposing raw `\Device\PhysicalMemory` or MSR read/write IOCTLS) and kernel debug/patching utilities. Also check `Get-WinEvent -LogName Microsoft-Windows-CodeIntegrity/Operational` for Event ID 3004/3077 (blocked

driver load attempts) covering the suspected infection window — a logged block attempt for a known-vulnerable driver around the time of this sample's compile timestamp (2026-07-04 12:14:58 UTC) would confirm the loader chain. Any signed driver found loaded that also appears on the blocklist should be treated as the loader and preserved for its own analysis, not deleted.

5. **Search for the PDB-referenced build path and dropped companion files:** `Get-ChildItem -Path C:\ -Recurse -Force -ErrorAction SilentlyContinue -Include hflar.exe,7.4.1.sys,CrackerDrv.pdb` and separately check `%AppData%\Roaming\reasonix\` for the `global-workspace\6.15\x64\Release\` build tree or any sibling directories — the PDB path implies a local build environment that may have left other artifacts (source, additional compiled variants) in the same tree. Preserve (copy, do not delete) anything found, and note its file timestamps for correlation against the compile time above.
6. **Check for registered registry/process filter altitudes matching this sample:** `fltmc filters` (for `minifilter/CmRegisterCallbackEx`-style registrations) and an EDR kernel-callback inventory if available, looking specifically for altitude strings 429628 or 166666 (Findings 3/3a and 4). Either altitude present and still registered means the driver (or a variant using the same altitudes) may still be loaded even if `driverquery` doesn't obviously flag it by filename — altitudes are a durable, hash-independent IOC per Detection Opportunities. If found, correlate the registering driver's image path back to steps 3-5 above.
7. **Hunt for the userland companion/controller process** implied by the `ObRegisterCallbacks` "grant full access to caller" logic — a process that would be protected/whitelisted by this driver, since the rootkit's value is in shielding a partner process, not existing in isolation. Concretely: from an elevated session, attempt `OpenProcess(PROCESS_TERMINATE, ...)` (e.g. via `taskkill` or Process Explorer) against every running process; any process that silently and consistently rejects termination/handle access from an otherwise-privileged caller is a candidate for the "protected" PID, and whatever process *can* freely open/manipulate it is a candidate for the "controller" PID (Finding 4's two paired global roles).
8. **Capture volatile process/handle state before remediation** if forensic preservation matters: take a full memory image with a kernel-aware tool (e.g., Magnet RAM Capture, WinPMem, or `DumpIt`) rather than relying on a live handle dump, since the "protected" process identified in step 7 is specifically designed to resist inspection by Process Explorer/Task Manager while running — a full memory image bypasses that resistance because it does not go through the `AccessMask`-filtering callback at all. Additionally capture `Get-Process | Export-Csv` and a Process Explorer handle-view screenshot as a supplementary live-triage record before the memory image (both quick to collect and useful for timeline correlation), but treat the memory image as the authoritative artifact for later `volatility`-based analysis of the protected/controller process pair.
9. **Re-image or restore from a known-clean backup** is the only fully reliable remediation for an active kernel-mode rootkit at this capability level; in-place cleanup cannot be trusted to evict kernel-resident code with page-table-level tampering capability fully.

Consolidated triage script (read-only checks from steps 1-6 above; run elevated; does not perform step 7's active `OpenProcess` probing, step 8's memory imaging, or step 9's remediation, each of which needs manual judgment/dedicated tooling):

```
# Requires: elevated PowerShell session.
$out = "C:\triage_$(Get-Date -Format yyyyMMdd_HHmmss)"
New-Item -ItemType Directory -Path $out -Force | Out-Null

# Step 1: DSE / test-signing state
bcdedit /enum "{current}" | Out-File "$out\bcdedit.txt"

# Step 2: Secure Boot status
Confirm-SecureBootUEFI | Out-File "$out\securebootuefi.txt"

# Step 3: Loaded drivers + fileless/self-deleted check
driverquery /v /fo csv | ConvertFrom-Csv | ForEach-Object {
    $_ | Add-Member -NotePropertyName BackingFileMissing `
        -NotePropertyValue (-not (Test-Path $_.Path)) -PassThru
} | Export-Csv "$out\loaded_drivers.csv" -NoTypeInfoation

# Step 4: Vulnerable-driver blocklist enforcement + blocked-load events
Get-CimInstance -Namespace root/Microsoft/Windows/CI -ClassName MS_VulnerableDriverBlocklist `
    | Out-File "$out\vulnerable_driver_blocklist_status.txt"
Get-WinEvent -LogName Microsoft-Windows-CodeIntegrity/Operational `
    -FilterXPath "[System[(EventID=3004 or EventID=3077)]]" -ErrorAction SilentlyContinue `
    | Export-Csv "$out\codeintegrity_blocked_loads.csv" -NoTypeInfoation

# Step 5: PDB-referenced build path / dropped companion files
Get-ChildItem -Path C:\ -Recurse -Force -ErrorAction SilentlyContinue `
    -Include hflar.exe, 7.4.1.sys, CrackerDrv.pdb `
    | Select-Object FullName, Length, CreationTime, LastWriteTime `
    | Export-Csv "$out\pdb_companion_files.csv" -NoTypeInfoation
Get-ChildItem -Path "$env:AppData\reasonix" -Recurse -Force -ErrorAction SilentlyContinue `
    | Select-Object FullName, CreationTime, LastWriteTime `
    | Export-Csv "$out\reasonix_build_tree.csv" -NoTypeInfoation

# Step 6: Registered registry/process filter altitudes
fltmc filters | Out-File "$out\fltmc_filters.txt"
Select-String -Path "$out\fltmc_filters.txt" -Pattern '429628|166666' `
    | Out-File "$out\fltmc_filters_flagged.txt"

Write-Host "Triage artifacts written to $out"
Write-Host "Review loaded_drivers.csv for BackingFileMissing=True and fltmc_filters_flagged.txt for altitude hits before proceeding to steps 7-9."
```

Detection Opportunities

These are derived directly from the confirmed static findings above; none were validated against a live/running instance of the driver (no dynamic analysis was performed — see Methodology). Treat as hunting hypotheses to test and tune in the target environment, not as ready-to-deploy production rules.

Registry-based (highest value — this is the driver's single trigger point, Finding 3a): - Alert on any `CmRegisterCallbackEx` registration with altitude string "429628", or on any `ObRegisterCallbacks` registration with altitude "166666" — both are non-standard, unassigned altitudes not associated with any known legitimate vendor. Tools that enumerate registered minifilters/callbacks (e.g., `fltmc`, or EDR kernel-callback inventories) should flag either altitude as an IOC by itself, independent of the sample's hash. - Because Finding 3a shows nearly every major capability (process-notify install, kbd/mouse recon, win32k table resolution, and the entire injection/token-theft toolkit) is gated behind *one* registry-callback dispatch point, instrumenting **all registry operations system-wide for a short window after this driver is confirmed loaded** (e.g., via `Reg` ETW provider or Sysmon Event ID 12/13/14) and correlating them against subsequent process/thread creation or token-reference activity is a plausible way to catch the actual trigger operation in the act, even without knowing its exact opcode encoding (see the unresolved selector-extraction question in Finding 3a).

Process/token-based (Findings 4 and 8): - Hunt for processes that **cannot be opened with** `PROCESS_TERMINATE`, `PROCESS_VM_OPERATION`, `PROCESS_VM_READ`, `PROCESS_VM_WRITE`, **OR** `PROCESS_SUSPEND_RESUME` **by a SYSTEM/Administrator-level caller** — this is Finding 4's AccessMask-stripping behavior and is anomalous under normal Windows operation (Task Manager/Process Explorer as admin can always open these rights on any process absent third-party interference). - Sysmon Event ID 8 (`CreateRemoteThread`) or Event ID 10 (`ProcessAccess`) correlated with a **kernel-mode caller** (i.e., no corresponding user-mode parent process shown as the source) is consistent with Finding 8's `KeStackAttachProcess/RtlCreateUserThread` primitive — standard EDR remote-thread detections are typically tuned for user-mode `CreateRemoteThread` callers and may miss a kernel-mode-attached equivalent. - Any unexplained, rapid privilege change on a low-integrity or unprivileged process (a process suddenly acting with SYSTEM-level token capabilities without a corresponding logon/impersonation event) is consistent with Finding 8's raw token-structure-copy primitive, which bypasses documented token-duplication APIs entirely and would not appear in a token-API audit trail.

File/driver-based: - Flag any `.sys/.exe`-named kernel driver whose backing file disappears from disk (`ZwDeleteFile`) shortly after `DriverEntry` while the driver remains loaded in `driverquery/Get-WindowsDriver` output — Finding 1's self-delete leaves the loaded-but-fileless state as a durable, queryable indicator even after the on-disk artifact is gone. - The static string/byte IOCs already listed in Indicators of Compromise (PDB path, debug string, driver-object name references, `ImpHash`, and the `fothk` section's byte pattern) are suitable for direct YARA/string-scan rules against other hosts or file repositories, though note none of them are surfaced at runtime in a way that's trivially observable without either a file to scan or kernel debugging access. - The non-standard `fothk` PE section (see Indicators of Compromise) — mostly `0xCC` filler with a single `jmp` stub — is itself a scannable structural IOC:

a YARA rule matching on a PE section named `fothk` with this fill/stub pattern, or generically flagging sections that are >99% `0xCC` with an embedded jump, would catch this sample and plausible variants without relying on the hash or PDB path.

Known gaps in this detection guidance: no concrete registry key/value pattern can be specified for the trigger operation itself (the opcode-selector derivation was not traced — see Finding 3a); no detection is proposed for the four “shellcode-staging” helpers under Finding 8 since their payload-write step wasn’t isolated; none of the above were tested against a live system or EDR product — they are static-analysis-derived hypotheses only; and none of the bullets above have been translated into an osquery query — if a fleet agent (osquery/Fleet/Kolide) is already deployed, the `file/yara` tables could run the `fothk/PDB-path` YARA rule below across many hosts at once rather than one host at a time, but no minifilter-altitude equivalent exists in osquery’s schema, so the registry-based bullets above would still need the manual `fltmcs filters` check.

Sample YARA rule (file/repository scanning — combines the static IOCs from bullet above; untested against other samples, may need tuning against benign `0xCC`-padded sections):

```
rule susp_kernel_rootkit_registrycallback_fothk
{
  meta:
    description = "Hunts for this sample or close variants via PDB path, debug string,
and non-standard fothk section"
    reference = "Internal FINDINGS_REPORT.md, static analysis only, not validated
against other samples"
    hash = "c7ec30029a10580a0b33b71b75d34efe053cc34d908bf75998f457a0c1f074f6"

    strings:
      $pdb =
"\\AppData\\Roaming\\reasonix\\global-workspace\\6.15\\x64\\Release\\CrackerDrv.pdb"
      $dbg = "[Driver] RegistryCallback: exception caught"
      $alt_registry = "429628" wide
      $alt_ob = "166666" wide
      $section_name = "fothk"

  condition:
    uint16(0) == 0x5a4d and
    (
      $pdb or $dbg or
      (($alt_registry or $alt_ob) and $section_name) or
      2 of ($pdb, $dbg, $alt_registry, $alt_ob, $section_name)
    )
}
```

Sample Splunk/SPL hunt queries (assumes Sysmon + a registry-filter/callback inventory feed; field names are illustrative — adapt to the actual source/sourcetype in your environment):

```
# Self-delete / loaded-but-fileless driver (Finding 1) – correlate a driver load
# with its own backing file being deleted shortly afterward.
index=sysmon (EventCode=6 OR EventCode=23)
| eval driver_path=coalesce(ImageLoaded, TargetFilename)
| transaction driver_path startswith=(EventCode=6) endswith=(EventCode=23) maxspan=5m
| where isnotnull(driver_path)
| table _time, driver_path, EventCode, Image

# Kernel-mode remote-thread injection (Finding 8, case 12/13) – CreateRemoteThread
# or ProcessAccess events with no plausible user-mode source process.
index=sysmon (EventCode=8 OR EventCode=10)
| where isnull(SourceImage) OR SourceImage="System" OR SourceImage="*ntoskrnl*"
| table _time, EventCode, SourceImage, TargetImage, GrantedAccess

# AccessMask-stripped process (Finding 4) – a high-privilege caller's OpenProcess
# request against a specific PID gets back a GrantedAccess missing the bits
# Finding 4 strips (0x1/0x8/0x10/0x20/0x800), despite an admin/SYSTEM caller.
index=sysmon EventCode=10
| eval expected_bits=tonumber("1FFFFFF",16)
| eval stripped_bits=tonumber("839",16)
| where (tonumber(GrantedAccess,16) BITAND stripped_bits) == 0 AND SourceUser IN
("SYSTEM","*Administrator*")
| table _time, SourceImage, SourceUser, TargetImage, GrantedAccess
```

Glossary

Altitude — A numeric string (e.g. "429628") that identifies a filter's position in a stack of registered kernel filters (registry filters via `CmRegisterCallbackEx`, process/thread filters via `ObRegisterCallbacks`). Microsoft assigns official altitude ranges to legitimate vendors; malware typically picks an arbitrary unassigned value, as this sample does.

AccessMask — A bitmask (`ACCESS_MASK`, a 32-bit value) requested when opening a handle to a kernel object, specifying which operations the caller intends to perform (e.g. terminate, read memory, write memory). The kernel grants a handle whose rights are the requested mask intersected with what the security check allows. `ObRegisterCallbacks` allows a driver to rewrite this mask in flight before the handle is returned to the caller.

Authenticode — Microsoft's code-signing format for Windows executables and drivers, embedding a cryptographic signature (and, typically, a timestamp and publisher identity) in the PE file. Its absence in this sample (`IMAGE_DIRECTORY_ENTRY_SECURITY` RVA/Size = 0) is the basis for calling it unsigned — see Sample Identification.

BYOVD (Bring Your Own Vulnerable Driver) — An attack technique where an adversary installs a legitimately signed but exploitable third-party driver (rather than their own unsigned code) specifically to abuse its vulnerabilities for kernel code execution, sidestepping Driver Signature Enforcement.

CR3 — An x86-64 control register holding the physical base address of the current process's top-level page-table structure (the PML4 table). Reading CR3 is normally done only by the memory manager itself; user- or kernel-mode software reading it directly is unusual and a common indicator of hand-rolled memory-management code.

CR8 — A separate x86-64 control register that holds the Task Priority Register (TPR) value, used for interrupt-priority (IRQL) management. Despite superficial opcode similarity to CR3 access, this register has nothing to do with paging.

capa — A static-analysis tool (Mandiant/FLARE) that automatically detects a binary's capabilities (e.g., process injection, persistence, anti-analysis) and maps them to MITRE ATT&CK techniques from patterns in its code. Did not produce usable output in this analysis environment (see Methodology) — the ATT&CK mappings in this report were derived manually instead.

ClamAV — An open-source, signature-based antivirus scanner. Referenced in this report only to note that its clean/no-detection result on this sample was against a signature database more than 7 days out of date, and is therefore not treated as a meaningful negative signal (see Assessment Limitations).

DeviceObject / DriverObject — Core Windows I/O Manager structures. A `DRIVER_OBJECT` represents one loaded driver and holds a linked list of the `DEVICE_OBJECTS` it has created; each `DEVICE_OBJECT` can further point to a "next lower" device in an attached device stack (used by filter drivers).

DriverEntry — The mandatory entry-point function of a Windows kernel driver, analogous to `main()` in a user-mode program. Called once by the I/O Manager when the driver is loaded.

DSE (Driver Signature Enforcement) — A Windows kernel policy that refuses to load kernel-mode drivers lacking a valid Authenticode (or Microsoft-issued) code signature, except when the system is booted in test-signing mode, or Secure Boot/DSE has been explicitly disabled.

EPROCESS — The internal, undocumented kernel data structure Windows uses to represent a running process (holding its PID, primary token pointer, handle table, and other process-level state). Kernel APIs like `PsLookupProcessByProcessId` return a pointer to this structure; code that reads or writes its fields directly (rather than through documented process APIs) is operating below the level normal drivers touch.

EDR (Endpoint Detection and Response) — Security software that monitors process, file, registry, and network activity on an endpoint for malicious behavior, typically relying on kernel callbacks and user-mode hooks similar to (and directly targeted by) this sample's own capabilities.

FLOSS (FLARE Obfuscated String Solver) — A static-analysis tool (Mandiant/FLARE) that recovers strings a malware sample constructs or decodes at runtime — e.g., by XOR-decoding or building strings character-by-character on the stack — which do not appear in a plain `strings` dump of the file.

fothk — A non-standard, 4,096-byte PE section in this sample, 4,091 of whose bytes are `0xcc` (`int 3`) filler with a single relative `jmp` stub at offset +16. Not a name or pattern produced by any standard compiler/linker; concluded to be a reserved hook/trampoline scratch region rather than an anti-debug check (see Sample Identification and Indicators of Compromise) — distinct from `0xcc` appearing as a breakpoint opcode in executable code paths (see Assessment Limitations).

ImpHash (Import Hash) — An MD5 hash computed over a PE file's imported function names and their order, used to cluster or identify related samples that share a compiler/toolchain/library fingerprint even when their code bodies differ.

IRQL (Interrupt Request Level) — A priority level the Windows kernel runs code at, controlling which interrupts and thread-scheduling activity can preempt it. Certain kernel operations (e.g., some paging or thread-attach primitives) can only run correctly at specific IRQLs; the CR8 access noted under CR8 above is an IRQL save, not a paging-related operation.

IOCTL (I/O Control Code) — The mechanism by which a user-mode application sends commands/data to a kernel driver, normally via `DeviceIoControl()` against a named device object the driver created with `IoCreateDevice`. This sample creates no such device, meaning it has no conventional userland control channel.

IRP (I/O Request Packet) — The structure Windows uses to represent an I/O operation (read, write, control code, etc.) passed down a driver stack. A "filter driver" intercepts IRPs by

attaching a device object above the target driver in the stack and inspecting/modifying IRPs as they pass through — the standard technique for kernel-level keylogging, which this sample's imports do not support (no `IoAttachDeviceToDeviceStack/IofCallDriver`).

kbdclass / mouclass / kbdhid / mouhid / i8042prt — The stock Windows keyboard and mouse driver stack. `i8042prt` (legacy PS/2) or `kbdhid/mouhid` (USB HID) sit at the bottom; `kbdclass/mouclass` sit above them as the “class driver” that presents a uniform interface to Win32 input APIs. Attaching a filter above `kbdclass/mouclass` is a classic technique for capturing all keystrokes/mouse movements system-wide.

MalwareBazaar — A public malware-sample-sharing and threat-intelligence platform (abuse.ch) where researchers upload and tag submitted samples, including community YARA rule matches. Consulted here for external corroboration only — see External Intelligence Corroboration.

Minifilter / registry filter — A driver component that registers to receive a callback on every operation of a given type (file I/O minifilters via `FltRegisterFilter`; registry operations via `CmRegisterCallbackEx`), allowing it to observe, block, or modify the operation before it completes.

NTSTATUS — The standard return-code type for Windows kernel APIs, a 32-bit signed value where negative values indicate failure (e.g. `0xC0000001` = `STATUS_UNSUCCESSFUL`). Disassembly comments throughout this report reference specific `NTSTATUS` values returned by traced functions.

Ob callback (`ObRegisterCallbacks`) — A kernel API letting a driver register pre/post-operation callbacks that fire whenever any process or thread handle is created, opened, or duplicated anywhere on the system — used legitimately by AV/EDR products for the same interception point this sample abuses to blind them.

PDB (Program Database) — A debug-symbol file produced by the Microsoft linker, whose full build path is often embedded in the compiled binary even in release builds; recovering it can reveal the developer's local file-system layout or project name (`CrackerDrv.pdb` in this sample).

PID (Process ID) — The unique numeric identifier Windows assigns to a running process for its lifetime.

Primary token — The kernel object (`TOKEN`) attached to a process that defines its full security identity: user/group SIDs, privileges (e.g., `SeDebugPrivilege`), and integrity level. All access checks the OS performs on that process's behalf are evaluated against its primary token. Copying another process's primary-token fields into a process's own token (as Finding 8 does) is equivalent to granting it that other process's complete security context, without going through any documented logon, impersonation, or token-duplication API.

PROCESS_ACCESS_MASK — The specific set of `ACCESS_MASK` bit values defined for process objects (e.g. `PROCESS_TERMINATE` = `0x1`, `PROCESS_VM_READ` = `0x10`, `PROCESS_VM_WRITE` = `0x20`, `PROCESS_SUSPEND_RESUME` = `0x800`), documented by Microsoft and used to determine what operations a given process handle permits.

PTE (Page Table Entry) — A single entry in the multi-level page-table hierarchy that maps a virtual memory page to a physical page frame and encodes its protection bits (readable/writable/executable/etc.). Directly editing PTE bits is a technique for bypassing memory protections, such as read-only kernel code pages, and is distinct from the physical-address *resolution* behavior actually observed in this sample (see Finding 5).

radare2 (r2) — An open-source command-line disassembler and binary-analysis framework, the primary tool used for all disassembly and cross-reference work in this report (see Methodology for the specific commands used).

Rootkit — Malware whose primary purpose is to hide the presence of itself or other malicious components (processes, files, registry keys, network connections) from the operating system and security tooling, typically by operating at a privilege level (kernel mode) below where detection tools run.

Secure Boot — A UEFI firmware feature that only allows cryptographically signed bootloaders and, by extension (via DSE), signed kernel drivers to execute during startup, closing off one common path to loading unsigned kernel code like this sample.

SEH (Structured Exception Handling) — The Windows mechanism for handling hardware faults and software exceptions during code execution. A function wrapped in SEH can catch a fault (e.g., an invalid memory access) and continue execution gracefully instead of crashing; Finding 3a's registry-callback dispatcher uses SEH to log and swallow any exception rather than crashing the kernel, which would otherwise bugcheck the entire system.

Subsystem (IMAGE_SUBSYSTEM_NATIVE) — A field in the PE optional header identifying what environment a binary expects to run in. Value 1 (Native) indicates a kernel-mode driver or native NT component, as opposed to 2/3 (GUI/Console) for ordinary Windows applications — the strongest single structural signal that this .exe-named file is actually a .sys driver.

TLB (Translation Lookaside Buffer) — A CPU cache of recent virtual-to-physical address translations. After manually editing a page table entry, software must issue `invlpg` (or a full TLB flush) to force the CPU to discard any stale cached translation for that address — the absence of any `invlpg` in this binary is part of why Finding 5 is scoped to address *resolution* rather than confirmed PTE *patching*.

UIPI (User Interface Privilege Isolation) — A Windows mechanism that prevents lower-integrity processes from sending window messages to higher-integrity ones (e.g., a standard user process cannot post messages into an elevated/admin window), intended to block one form of privilege-escalation via UI automation.

VirusTotal (VT) — A public multi-engine file-scanning and threat-intelligence platform that aggregates detection results from dozens of antivirus vendors, plus community comments, YARA matches, and (as used here) an automated AI code-summary feature. Consulted here for external corroboration only — see External Intelligence Corroboration for how each VT claim was weighed against direct disassembly.

WDAC (Windows Defender Application Control) — A Windows policy engine that can allow-list or block-list specific drivers/applications from running, including enforcement of Microsoft’s vulnerable-driver blocklist (referenced in Recommendation 4). Unlike EDR, WDAC is a preventive control-execution gate, not a detection/response product.

Win32k / `w32pServiceTable` — `win32k.sys/win32kfull.sys` is the kernel-mode component implementing the Windows GUI and USER/GDI subsystems. `w32pServiceTable` is its internal (non-exported) system-call dispatch table; locating it via signature scanning — rather than using documented user-mode GDI/USER APIs — lets kernel-mode code invoke GUI operations directly, making them invisible to the user-mode API hooks most security products rely on.

YARA — A pattern-matching rule format and tool widely used to identify malware families or code characteristics via byte/string signatures. YARA rule hits referenced in this report (from VirusTotal and MalwareBazaar) are third-party pattern matches against this sample, not this analyst’s own rules — see External Intelligence Corroboration for how each is weighed.

Appendix: Methodology

Analysis type: Purely static. The sample was never executed, in a sandbox or otherwise — all findings in this report derive from disassembly, string/import inspection, and PE structural analysis of the file as received. No dynamic analysis (debugger single-stepping, kernel-mode tracing, or live execution of any kind) was performed at any point in this investigation.

Primary tool: radare2 (`r2`), invoked non-interactively per analysis step via `r2 -q -e scr.color=0 -e bin.relocs.apply=true -e asm.pseudo=true -c '<commands>' <sample>`. Because `r2`’s analysis state does not persist across separate invocations, every command chain that depended on function-boundary or cross-reference analysis re-ran `aaa` (or a targeted `af @ <addr>`) before any `pdf/axt/pD` call in the same invocation. Key `r2` commands used throughout: `aaa` (whole-binary auto-analysis), `af1` (list analyzed functions, used to find true function-start addresses when `af` anchored to a call site produced a truncated stub), `pdf @ <fcn>` (full function disassembly), `axt @ <addr>` (find callers/cross-references to an address), `pD <N> @ <addr>` (linear disassembly, used to disambiguate case-dispatch ordering when `pdf`’s control-flow rendering was ambiguous), and `/r <symbol>` (enumerate all reference sites to an imported symbol, used to find every call site for security-relevant imports such as `RtlCreateUserThread`, `KeStackAttachProcess`, `ZwAllocateVirtualMemory`, `PsReferencePrimaryToken`, and `ObReferenceObjectByHandleWithTag`).

Supporting tools: - **FLOSS** (Mandiant/FLARE Obfuscated String Solver) — used to recover runtime-constructed/decoded strings absent from a plain `strings` dump, notably the `win32k` syscall names underlying Finding 6. - **ClamAV** — run against the sample; returned a clean result, but the signature database was more than 7 days out of date at scan time and this result is explicitly not treated as a meaningful negative signal (see Assessment Limitations). - **capa** — attempted but did not produce usable output in this environment (embedded default rule set was missing and was not substituted with an external rules repository). All MITRE

ATT&CK technique mappings in this report are analyst-derived from manual reverse engineering, not capa output. - **VirusTotal / MalwareBazaar** — consulted for external corroboration only (vendor detection labels, community YARA hits, an automated AI code-summary feature, alternate-filename history) and treated throughout as third-party claims to be checked against direct disassembly, not as primary evidence — see External Intelligence Corroboration for the specific disposition of each externally-sourced claim.

Verification approach: Where this report states a function was “fully traced” or gives a confidence rating of High, that specifically means the function’s boundaries were confirmed via `afl/aaa` (not assumed from a call-site anchor), its full instruction sequence was read end-to-end via `pdf`, and any claimed call targets or cross-references were independently confirmed via `axt/pd` rather than inferred from function naming, symmetry with another function, or partial disassembly. Several findings in this report (see Appendix: Corrections Made During Audit) were revised specifically because an earlier pass fell short of this bar — e.g., assuming a symmetric helper was shared/reused between the keyboard and mouse paths (Finding 7) without independently disassembling the mouse-side call target, which turned out to be a separate function.

Known methodological gaps: - No formal function-by-function inventory of the binary’s complete symbol/function population was produced — only functions reached via cross-reference from `DriverEntry`, the two registered callbacks, or an explicit import search were disassembled and dispositioned in this report. Functions with no traceable path to a documented capability may exist and go uncounted. - No opcode/byte-pattern search was performed across the driver’s *executable code paths* for `int 3/0xCC` breakpoint-style anti-debug checks, timing-based anti-analysis, or VM/sandbox-detection logic; the presence or absence of such checks in code was not assessed. (This is distinct from the `fothk` section’s `0xCC` fill bytes noted in Indicators of Compromise, which is a PE section-level structural observation, not a code-path anti-debug check, and was concluded to be a hook/trampoline scratch region — see Sample Identification.) - No attempt was made to statically emulate or symbolically execute the registry-callback opcode-selector-extraction logic (see Finding 3a) to enumerate the set of registry operations that would reach each dispatcher case.

Appendix: Corrections Made During Audit

This report supersedes an earlier working version produced during this investigation. Material corrections from that version:

1. **CR3/page-table manipulation:** went through two corrections. First concluded “not substantiated” based on an incomplete opcode-pattern sweep; a follow-up pass located a genuine `mov rcx, cr3` at `0x140008dd6` and over-corrected to claim a manual PTE walk plus `invlpg`. A final exhaustive verification found **no `invlpg` anywhere in the binary** and confirmed the surrounding code calls the documented `MmGetVirtualForPhysical/MmGetPhysicalMemoryRangesEx` APIs rather than hand-patching page-table entries. Current finding (Finding 5): the CR3 read is real and singular, but

its role is physical address resolution/reconnaissance, not a confirmed write-protection bypass.

2. **Keyboard/mouse address-range globals:** previously reported as a single shared pair and corrected to two distinct pairs — mouse: 0x140013040/0x140013050; keyboard: 0x140013048/0x140013058.
3. **Added finding:** PsSetCreateProcessNotifyRoutineEx import and invocation, previously undocumented.
4. **Added finding, later fully resolved as Finding 3a:** an internal opcode dispatcher gating major capability installation, previously undocumented and only vaguely characterized. Direct disassembly of fcn.140006254 (the function registered as the CmRegisterCallbackEx notify routine at 0x14000606d) confirmed it is a single 764-instruction, 92-branch dispatcher, and that the PsSetCreateProcessNotifyRoutineEx call (item 3 above) and the keyboard/mouse reconnaissance trigger (Finding 7) are opcode-gated branches fired from *inside* this one registry-callback function — not independent installs at DriverEntry time as the flow diagram previously depicted. Finding 3 and the flow diagram were both updated to reflect this.
5. **Added IOC:** PDB build path and an embedded debug string, previously unextracted.
6. **Added caveat:** ClamAV's clean result was against a stale signature database and should not be read as a negative signal; this was previously presented without qualification.
7. **Added caveat:** capa did not run successfully in this environment; this tooling gap was previously undisclosed.
8. **Mouse-side list-walk helper independently disassembled and corrected:** previously assumed (by symmetry, not direct evidence) that the keyboard-path helper fcn.14000c0e4 was shared/reused for the mouse path. Direct disassembly identified the actual mouse-side call target as a **separate, structurally identical function**, fcn.14000c208, called from fcn.14000bf94 at 0x14000c0ab. Finding 7 now includes the annotated disassembly of fcn.14000c208 and confirms it writes to 0x140013040/0x140013050, matching the address-range globals already recorded for the mouse path — this independently corroborates (rather than changes) the address-range-global correction from item 2 above, this time from mouse-side evidence rather than keyboard-side.
9. **W32pServiceTable resolution fully disassembled — corrected to show it is a separate primitive from Finding 2's module resolver:** the report previously described the win32k table as located by "a signature scan" without further detail, leaving it ambiguous whether this reused the PsLoadedModuleList-walk resolver documented in Finding 2 (fcn.140006114). Direct disassembly of the full four-function chain (fcn.140001218 → fcn.140001090 → fcn.1400010f8/fcn.140001170/fcn.140001030) confirmed it is a **distinct, generic masked byte-pattern-scan primitive** — it resolves win32kfull.sys's base via its own internal call to fcn.1400010f8 (a PsLoadedModuleList walker structurally similar to, but a separate function from, Finding 2's fcn.140006114), locates the .text section by name, then scans it with an explicit "x????xxxxx" mask against pattern bytes e8 00 00 00 00 8b f8 85 c0 75 0e, and resolves the matched instruction's call re132 operand to an absolute address. Finding 6 now includes this full annotated disassembly. (The flow diagram briefly showed this as a parallel DriverEntry-time branch before item 10 below

established it is actually opcode-gated via dispatcher case 16 — see item 10 and the flow diagram’s current placement.)

10. Finding 8 (process injection/token toolkit) fully traced and re-ranked as the top severity

finding: previously listed only as unconfirmed imports (RtlCreateUserThread, MmCopyVirtualMemory, PsReferencePrimaryToken, etc.) with no call sites identified, at Confidence: Medium. Direct disassembly located and traced every call site:

fcfn.14000741c (remote thread injection via PsLookupProcessByProcessId →

KeStackAttachProcess → RtlCreateUserThread → KeUnstackDetachProcess), fcn.140008148

(resolves LoadLibraryA in the target process and chains into the thread-injection

primitive — a LoadLibraryA-based DLL-injection loader), and fcn.1400049b4/fcn.140005bdc

(validates a target PID is still alive, then fans out into 8 subroutines including a raw 32-byte primary-token content swap between two processes — a

token-theft/privilege-escalation primitive). Critically, all three call chains were found to be additional case targets (12, 13, 14) within the *same* first-tier opcode dispatch cascade documented in Finding 3a (fcfn.140006254). I.e., this entire toolkit is not freestanding code but is opcode-gated inside the registry callback, exactly like Findings 6/7’s triggers. This also revealed that dispatcher case 16 invokes Finding 6’s win32k table-scan resolver (fcfn.140001218), meaning that resolver is opcode-gated too, not purely load-time. Finding 8, Finding 3a, the Executive Summary, the Key Findings ranking, and the flow diagram were all updated to reflect this; Finding 8 moved from rank 8/Medium confidence to rank 1/High confidence given its remote-triggerable RCE and privilege-escalation impact.

11. Finding 2 corrected to remove a stale claim left over from before item 9’s win32k

correction: Finding 2’s body text still asserted that fcn.140006114 (the generic module resolver) “locates win32kfull.sys and pattern-scans it for W32pServiceTable” — directly contradicting item 9 above and Finding 6, which established this is done by a separate four-function chain (fcfn.140001218, etc.) that only reuses one structurally similar (not identical) sub-helper. Finding 2 now correctly points to Finding 6’s chain instead of claiming the win32k scan as its own.

12. Finding 3a’s case-1/Finding 4 link downgraded from stated fact to unverified

candidate: Finding 3a’s case-1 description (and the flow diagram) asserted that case 1’s tracked-target caching (0x140012b50/0x140012b48) “feeds” Finding 4’s ObRegisterCallbacks AccessMask logic — but Finding 4’s four AccessMask-gating globals are at entirely different addresses (0x140012b80/b88/b90/b98), and no call/data-flow link between the two was directly traced. This overstated a claim that Assessment Limitations already correctly hedges as “plausible but unverified.” Finding 3a and the flow diagram were both reworded to match that hedge.